



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Food Rescue Connect: Smart Food Donation and Redistribution Management System

Industry Context

A CASE STUDY REPORT

Submitted in the partial fulfilment for the Course of

CSA1016-Software Engineering

to the award of the degree of

BACHELOR OF ENGINEERING

IN

CSE-AI

Submitted by

Sk. Ashwaq Ahamed (192572070)

Under the Supervision of

Dr. K. ANITA DAVAMANI

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

1. Problem Overview and Implemented Solution

FoodRescue Connect is a smart food donation and redistribution management system intended to reduce avoidable food waste and improve access to surplus edible food. The system connects donors such as restaurants, supermarkets and hotels with NGOs and volunteers. The platform records donation details and supports the movement of food from availability through pickup and delivery.

The problem arises when surplus edible food is discarded because donation collection is handled through slow or manual communication. Delays can cause food to spoil, while NGOs may not have timely information about nearby donations. FoodRescue Connect addresses this coordination problem through a centralized application, structured donation records, status tracking and a simple priority mechanism based on expiry time.

Key Functionalities

- Donor registration and login.
- Food donation creation with food name, quantity, expiry time and location.
- NGO view and acceptance of available donations.
- Volunteer pickup and delivery status updates.
- Priority assignment for donations approaching expiry.
- Dashboard for donation and delivery activity.
- Repository-based collaborative development using Git.
- Containerized execution using Docker and Docker Compose.

Review Scope

The supplied assessment asks the reviewer to explain the problem and solution, evaluate repository organization, inspect code readability and modularity, analyze Git history and branching, demonstrate testing, evaluate documentation, and provide findings and recommendations. These areas are covered throughout this report.

Review Objective

The main objective of the review is not only to identify whether the application works, but also to determine whether its codebase is understandable, maintainable, testable and suitable for collaborative software development.

2. Repository Organization Evaluation

A well-organized repository reduces the time required to locate functionality, understand dependencies and review changes. For FoodRescue Connect, the recommended organization separates the user interface, backend services, tests and deployment configuration.

Recommended Repository Organization

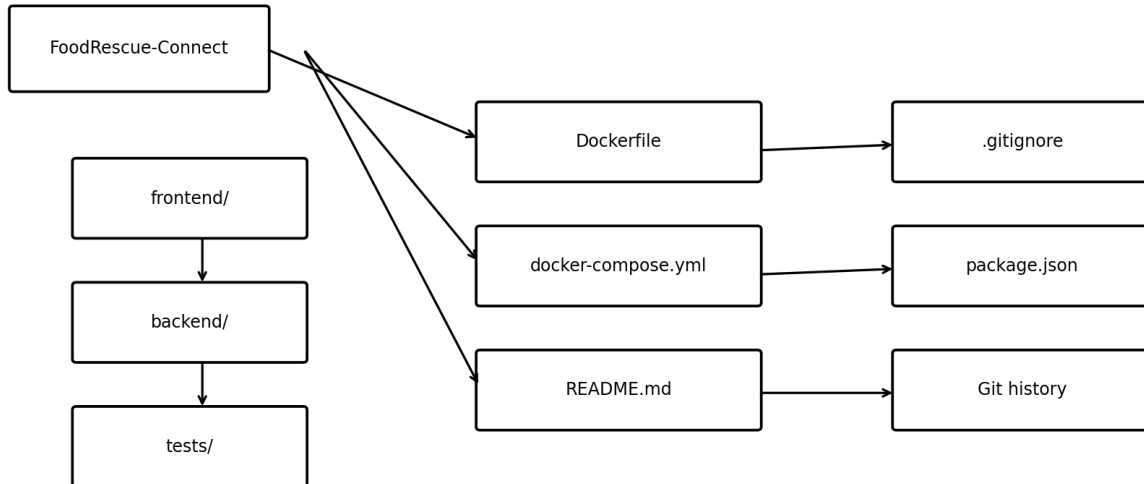


Figure 1. Recommended repository organization for code review

Recommended Structure

```
FoodRescue-Connect/  
├── frontend/  
│   ├── index.html  
│   ├── login.html  
│   ├── dashboard.html  
│   ├── donation.html  
│   ├── css/  
│   └── js/  
├── backend/  
│   ├── server.js  
│   ├── routes/  
│   ├── controllers/  
│   ├── models/  
│   └── package.json  
├── tests/  
├── Dockerfile  
├── docker-compose.yml  
├── .gitignore  
└── README.md
```

Evaluation of Naming and Organization

Area	Review Observation	Evaluation
Folders	Frontend, backend and tests are separated.	Good separation of concerns.
Routes	API endpoints are grouped under routes/.	Supports modularity.
Controllers	Request handling is separated from routing.	Improves maintainability.
Models	Database structures are grouped together.	Easy to locate data definitions.
Tests	Dedicated tests/ folder is recommended.	Makes validation discoverable.
Configuration	Docker and package configuration are visible at root.	Easy for new contributors.
README	Project setup and usage should be documented.	Essential for collaboration.

Maintainability Assessment

The structure supports maintainability because a contributor can work on a feature without searching through one large source file. Separating routes, controllers and models also reduces the likelihood that UI, database and business logic become tightly coupled.

3. Code Quality Review

The code review focuses on readability, modularity, coding standards, documentation and maintainability, which are major evaluation areas in the supplied assessment rubric. The strongest implementation should use small functions, meaningful names, consistent formatting and clear separation of responsibilities.

Readability Review

- Use descriptive names such as donation, expiryTime, donorId and deliveryStatus rather than single-letter variables.
- Keep functions focused on one operation, such as createDonation() or updateDeliveryStatus().
- Use consistent indentation and formatting across JavaScript files.
- Avoid large blocks of commented-out code.
- Add short comments only where business rules are not obvious.

Modularity Review

The backend should avoid putting all API logic inside server.js. Routes should identify endpoints, controllers should process requests, and models should define persistent data. This makes individual modules easier to test and modify.

Representative Review Example – Improvement

```
// Less maintainable example
app.post("/donation", async (req, res) => {
  // validation + database + priority + response
});

// Recommended structure
router.post("/donations", createDonation);

async function createDonation(req, res) {
  const donation = validateDonation(req.body);
  const priority = calculatePriority(donation.expiryTime);
  const saved = await Donation.create({ ...donation, priority });
  return res.status(201).json(saved);
}
```

The second example separates routing from business logic and makes the operation easier to test. In the final project, the exact implementation should be checked against this principle.

Coding Standards

- Use camelCase consistently for JavaScript variables and functions.
- Use PascalCase for model/class names when applicable.
- Keep API responses consistent.
- Use meaningful HTTP status codes.
- Avoid hard-coded credentials or database passwords in source files.
- Use environment variables for deployment-specific configuration.

4. Code Review Findings with Examples

Code Review Evaluation Flow

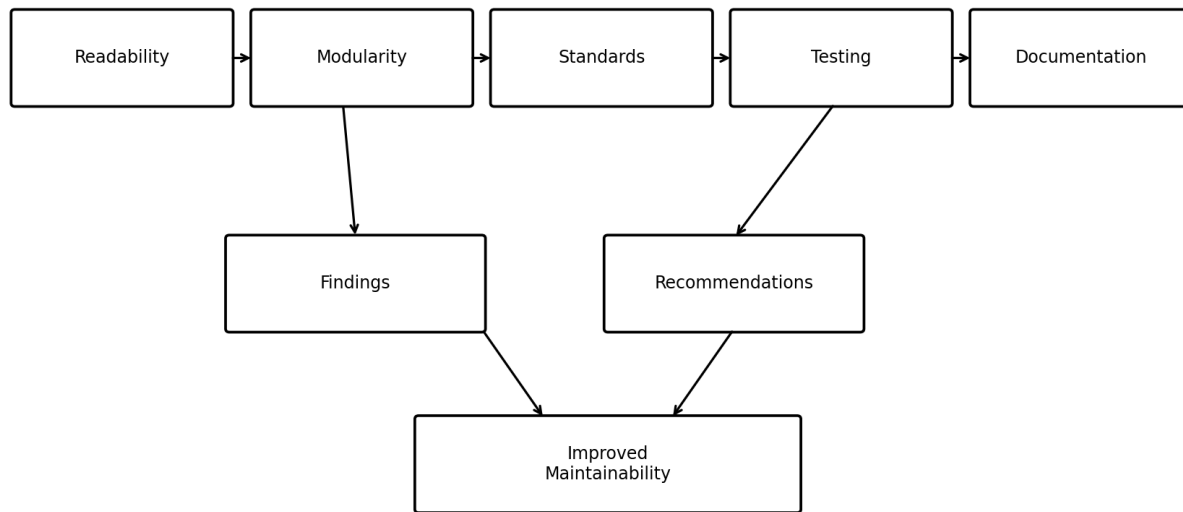


Figure 2. Code review evaluation flow

Strengths Identified

Finding	Impact	Assessment
Modular frontend/backend design	Reduces coupling and supports independent changes.	Strong
Role-oriented workflow	Maps system actions to donor, NGO and volunteer responsibilities.	Strong
Clear donation lifecycle	Available → Accepted → Picked Up → Delivered is easy to understand.	Strong
Simple priority logic	Provides an understandable smart feature for urgent donations.	Good
Git-based development	Supports traceability and collaboration.	Strong
Docker deployment	Improves portability of the application environment.	Good

Areas for Improvement

Area	Observation	Suggested Improvement
Validation	Donation inputs may contain missing or invalid values if validation is incomplete.	Add reusable validation functions.
Error Handling	Unexpected database/API failures need controlled responses.	Add centralized error middleware.
Security	Credentials should not be embedded in source code.	Use environment variables and secure authentication.
Testing	Each API path should have positive and negative test cases.	Add automated unit/API tests.
Logging	Important failures should be traceable.	Use structured application logging.
Documentation	Setup steps should be clear for a new contributor.	Expand README with prerequisites and commands.

Representative Code Review Observation

```
if (!foodName || !quantity || !expiryTime) {  
    return res.status(400).json({ error: "Invalid donation" });  
}
```

Observation: the validation is useful but can become repetitive if copied across multiple endpoints.
Recommendation: move validation into reusable functions or middleware so that the same rules are consistently applied.

5. Version Control Evaluation

The assessment requires analysis of commit history, branching strategy, commit messages and repository maintenance practices. Git is particularly important for FoodRescue Connect because different features can be developed independently and later integrated.

Recommended Branching Model

```
main
|
+-- develop
    |
    +-- feature/donation
    +-- feature/delivery
    +-- feature/dashboard
    +-- feature/testing
    |
    +-- release
```

Commit History Review

Example Commit	Review
Initial project structure	Appropriate starting point.
Add donor registration	Specific feature-oriented message.
Implement donation API	Clearly identifies the change.
Add NGO dashboard	Useful for tracing functionality.
Add volunteer delivery module	Describes a complete feature.
Add expiry priority logic	Identifies business-rule addition.
Add API tests	Shows testing activity.
Improve README setup	Documents repository maintenance.
Add Docker Compose configuration	Identifies deployment change.

Good vs Poor Commit Messages

Type	Example
Good	Add donation validation
Good	Fix delivery status update
Good	Add API tests for donation endpoint
Poor	changes
Poor	final
Poor	update

Repository Maintenance Review

- Keep main stable and use feature branches for changes.
- Use pull requests when collaborating with other contributors.
- Review changes before merging.
- Keep .gitignore updated to exclude node_modules, local environment files and generated artifacts.
- Do not commit passwords, API keys or database credentials.
- Use tags or release branches for submission versions.

6. Code Testing and Validation

The assessment requires evidence that the application functions correctly through appropriate testing, including test cases, execution results and screenshots. Testing for FoodRescue Connect should cover normal user actions as well as invalid inputs and status transitions.

Functional Test Cases

ID	Test Case	Input / Action	Expected Result
TC01	Open application	Open frontend URL	Homepage loads.
TC02	User login	Valid donor credentials	Donor dashboard opens.
TC03	Create donation	Valid food, quantity, expiry and location	Donation is saved.
TC04	Invalid donation	Missing food name	Validation error is shown.
TC05	Priority calculation	Expiry < 2 hours	Priority becomes HIGH.
TC06	NGO acceptance	Accept available donation	Status becomes Accepted.
TC07	Volunteer pickup	Accept pickup task	Status becomes Picked Up.
TC08	Delivery	Mark delivery complete	Status becomes Delivered.
TC09	Dashboard	Open dashboard	Statistics are displayed.
TC10	API failure	Database unavailable	Controlled error response.

Validation Approach

1. Run the backend and database.
2. Open the frontend and execute the donor flow.
3. Create a valid donation.
4. Check that the record appears in the dashboard/database.
5. Test an invalid input and confirm validation.
6. Use an NGO account to accept the donation.
7. Use a volunteer account to update pickup and delivery.
8. Confirm final status and dashboard counts.
9. Record the results and capture screenshots.

Test Result Summary Template

Test Group	Passed	Failed	Remarks
Authentication	_____	_____	_____
Donation Management	_____	_____	_____
Priority Logic	_____	_____	_____
NGO/Volunteer Workflow	_____	_____	_____
Dashboard	_____	_____	_____

7. Repository Documentation Evaluation

The assessment asks for evaluation of README completeness, installation instructions, usage guidance and dependencies. Documentation is a key part of maintainability because a new contributor should be able to understand and run the project without relying entirely on the original developer.

README Should Contain

- Project title and purpose.
- Problem statement and major features.
- Technology stack.
- Prerequisites such as Node.js and Docker.
- Repository setup instructions.
- Environment variable instructions.
- Database configuration.
- Commands to install dependencies.
- Commands to start the application.
- Testing commands.
- Expected application URL/ports.
- Folder structure explanation.
- Contribution and Git workflow guidance.

Recommended README Example

```
# FoodRescue Connect

## Overview
Smart food donation and redistribution management system.

## Setup
1. Clone the repository.
2. Install dependencies.
3. Configure environment variables.
4. Start MongoDB / Docker services.
5. Start the backend.
6. Open the frontend.

## Test
Run the project test command and review the results.

## Main Modules
- Donor
- NGO
- Volunteer
- Admin
- Donation priority
- Delivery tracking
```

Documentation Findings

Documentation Area	Current/Target Quality	Recommendation
README	Good foundation	Add complete setup and troubleshooting.
Installation	Should be step-by-step	Include Node.js, Docker and database prerequisites.
Usage	Should explain user roles	Add example donor → NGO → volunteer flow.
Dependencies	package.json should list them	Document important runtime dependencies.
Testing	Needs reproducible commands	Add exact test command and expected output.
Deployment	Docker should be documented	Add docker compose commands and ports.

8. Overall Findings and Recommendations

Overall Review Summary

FoodRescue Connect has a suitable modular concept for a full-stack academic project. The donor, NGO and volunteer workflow provides clear functional boundaries, while the repository can be organized into frontend, backend, tests and deployment configuration. Git provides traceability and Docker supports consistent execution. The major improvement opportunities are automated testing, centralized validation and error handling, security configuration, documentation and disciplined repository maintenance.

Review Category	Strength	Priority Improvement
Repository Organization	Logical separation of major components.	Keep naming consistent and document structure.
Code Quality	Modular design is achievable.	Reduce duplication and add reusable validation.
Version Control	Feature branches and meaningful commits support collaboration.	Use pull requests and protect main.
Testing	Clear functional workflow exists.	Automate API/unit tests and add negative cases.
Documentation	README can describe project basics.	Add full setup, usage and troubleshooting.
Maintainability	Separation supports future changes.	Add logging, configuration management and CI.

Recommendations

- Introduce automated unit tests for priority logic and validation.
- Add API integration tests for donation and delivery endpoints.
- Use centralized error-handling middleware.
- Add input validation middleware for all public endpoints.
- Use environment variables for database URLs and application secrets.
- Add authentication and role-based authorization to protected endpoints.
- Use pull requests and code review before merging to main.
- Protect main branch and require successful tests before merge.
- Improve README with installation, usage, testing and troubleshooting sections.
- Add CI workflow to automatically run tests for every pull request.

Future Code Improvements

- Replace simple expiry rules with a configurable priority service.
- Add demand prediction using historical donation data.
- Integrate GPS-based route optimization.
- Add notification service for urgent donations.
- Introduce audit logs for important status changes.
- Add performance monitoring and structured logs.
- Move toward cloud deployment when the application is ready for production use.

9. Conclusion

Conclusion

The Code Review and Repository Evaluation of FoodRescue Connect shows that the project can be structured as a maintainable full-stack application with clear separation between frontend, backend, data models and supporting services. The system's food donation lifecycle is simple enough to understand while still demonstrating meaningful software engineering concepts such as modularity, version control, testing, documentation and deployment.

The review also identifies practical improvements. Reusable validation, centralized error handling, automated tests, secure configuration, structured logging, better documentation and disciplined Git practices would improve the quality of the project. These recommendations directly support long-term maintainability and collaborative development.